# Designing your own misclassification model

## Contents



The models this package ships with — record linkage, conditional independence, fully non-parametric — are a starting menu, not a description of your data. Occupational coding drift does not look like name-matching failure. Age heaping does not look like either. If you use a model whose shape is wrong, you will get a confident, precise, wrong correction.

This vignette is about designing the right shape for your own setting. It walks through four steps: enumerate the error channels, turn them into a parameterized matrix, write that matrix as an R function, and verify it on simulated data before it touches anything real. It then shows how to use a large language model as a drafting assistant for step three, which is the tedious one — and, at more length, how to check what the assistant gives you. The division of labour is the whole point: **the assistant drafts, you verify.** An assistant that produces a plausible-looking matrix with a transposed index will cost you more time than writing it yourself, unless you have a habit of checking.

### The object you are designing

$\Delta$ is the distribution of what you *observe* given what is *true*:

$$\Delta_{y^*,\,(y_1,y_2)} \;=\; \Pr(Y_1 = y_1,\, Y_2 = y_2 \mid Y^* = y^*).$$

Concretely, it is a `J` by `J^2` matrix, stored in the package as a plain numeric vector of length `J^3`. Three facts about the layout are load-bearing and are where hand-written (and generated) code goes wrong:

- **Rows are the truth.** Row `y*` of $\Delta$ is a probability distribution over the `J^2` possible pairs of recorded values. Rows sum to one.
- **Columns are pairs, with the first measure varying fastest.** Column `(y2 - 1) * J + y1` holds $\Pr(Y_1 = y_1, Y_2 = y_2 \mid Y^*)$.
- **The vector is column-major**, so `matrix(Delta, nrow = J)` recovers the matrix.

When the two measures err independently given the truth — the usual case, and the assumption the package's built-in models make — you never build the `J` by `J^2` object directly. You build two small `J` by `J` matrices,

$$\Delta^{(m)}_{y_m,\,y^*} = \Pr(Y_m = y_m \mid Y^* = y^*), \qquad m = 1, 2,$$

each with **columns** that sum to one, and combine them. Note the transposition: in $\Delta^{(m)}$ the truth indexes columns; in $\Delta$ it indexes rows. This one idiom does the combining, and it is worth copying verbatim rather than re-deriving:

```r
combine_Delta <- function(Delta1, Delta2) {
  blocks <- lapply(1:nrow(Delta2), function(y2) diag(Delta2[y2, ]) %*% t(Delta1))
  c(do.call(cbind, blocks))
}
```

## Step 1: enumerate the error channels

Before writing any code, write down — in words — every way a value in your outcome column could differ from the truth. In historical microdata the list is usually short and drawn from four families.

**Transcription error.** A clerk, a keyer, or an OCR engine misreads the source. Errors are *local in the written form*: "Baker" becomes "Barker", 1863 becomes 1868. They are not local in the analytic ordering — a transcription error in an occupation string can land anywhere in the income distribution. Shape: a diffuse matrix with a strong diagonal, with off-diagonal mass concentrated on categories with similar spellings or codes.

**Coding drift.** The same underlying job is coded differently in different years, or by different agencies, or under a revised classification. Errors are *systematic, not random*: every "agricultural labourer" in one source becomes "farm hand" in another. Shape: mass concentrated in a few specific off-diagonal cells, often asymmetric, and often stable across the distribution of $X$. Crucially, this channel violates the "modal report is the truth" condition if the drift is large enough to move the mode.

**Record-linkage failure.** The name-matching algorithm links the wrong two people. The recorded outcome is then a stranger's outcome, drawn from whatever pool of people the algorithm was choosing among. Shape: the "slab" — a fraction $\alpha$ of each column replaced by a common distribution $\rho$ that does not depend on the truth.

**Heaping.** Respondents and enumerators round. Ages pile on multiples of five, incomes on round hundreds, birth years on decades. Shape: mass flows *toward* focal categories and away from their neighbours. Heaping is the channel most likely to surprise you, because it is not mean-reverting and can bias a slope in either direction.

Two questions to ask about each channel you list:

1. *Does it depend on the truth?* Transcription and heaping do. Linkage failure, to a first approximation, does not — which is why it is the easiest to parameterize.
2. *Does it depend on the regressor?* If yes, you must condition on something that makes it not depend on the regressor, using the `controls` argument of `prep_misclassification_data()`. The model assumes errors are independent of $X$ given $Y^*$ (and given the controls).

## Step 2: turn channels into a parameterized $\Delta$

Each channel maps to a functional form with a handful of parameters. Three forms cover most of what applied work needs.

### The record-linkage slab

$$\Delta^{(m)} = (1 - \alpha_m)\, I + \alpha_m\, \rho_m \mathbf{1}',$$

read as: with probability $1 - \alpha_m$ the link is right and you observe the truth; with probability $\alpha_m$ it is wrong and you observe an independent draw from $\rho_m$. Parameters: one rate $\alpha_m$ per measure, plus the $J - 1$ free elements of $\rho_m$ if you estimate it rather than plugging in a known pool distribution. This is `model_to_Delta_RL_ind()`, and the factories `make_empirical_Delta_RL*()` build variants that plug in the empirical margin for $\rho$.

Which $\rho$? Not, in general, the unconditional marginal distribution of the outcome. A failed link draws from the pool of candidates the algorithm was choosing among — people with similar names, similar birth years,

and *in the same place*, if the linker matched on place. Using the unconditional margin when the true pool is narrower makes false links look like correct ones and biases $\hat{\alpha}$ downward. The last section of this vignette shows how to handle this properly.

#### Monotone and ordinal forms

When the outcome is ordered — an occupational rank, an income decile, a literacy scale — errors usually move you a short distance along the ordering, not to an arbitrary category. The natural form is a band matrix:

$$\Delta^{(m)}_{y_m,y^*} = \begin{cases} p^{\text{down}} & y_m = y^* - 1 \\ 1 - p^{\text{down}} - p^{\text{up}} & y_m = y^* \\ p^{\text{up}} & y_m = y^* + 1 \\ 0 & \text{otherwise,} \end{cases}$$

with the boundary columns renormalized. Two parameters per measure describe the entire matrix regardless of `J`, which is why ordinal designs stay tractable when `J` is large. Richer variants let the rates vary with $y^*$, or widen the band to two steps.

#### Heaping forms

Fix a set of focal categories $F$ (multiples of five, round hundreds, decades). With probability $h$ the reported value is snapped to the nearest focal category; otherwise it is correct:

$$\Delta^{(m)}_{y_m,y^*} = (1 - h_m)\,\mathbf{1}\{y_m = y^*\} + h_m\,\mathbf{1}\{y_m = f(y^*)\},$$

where $f(y^*)$ is the nearest element of $F$. One parameter per measure. Note that this matrix has zero columns off the diagonal and the focal column, and that it is diagonally dominant only while $h_m < 1/2$.

#### Composing channels

Channels compose by matrix multiplication. If the truth is first mis-transcribed ($T$) and the resulting record is then mis-linked ($L$),

$$\Delta^{(m)} = L\,T,$$

and because each factor has columns summing to one, so does the product. Compose in the order the errors physically happened. Do not add channels — adding two column-stochastic matrices does not give you one.

## Step 3: write the closure

`misclassifyr()` takes `model_to_Delta` as a **function of a single unconstrained numeric vector** `psi`. The contract:

| Requirement | Why |
|---|---|
| Accepts one argument, `psi`, any real vector | The optimizer works in unconstrained space |
| Returns `c(Delta)`, a numeric vector of length `J^3` | The likelihood indexes it directly |
| Rows of `matrix(Delta, nrow = J)` sum to one | It is a conditional distribution |
| All entries non-negative | Same |
| `J` recoverable from `length(psi)`, or captured in the closure | The function is called with `psi` alone |

The last row is the reason these are written as *closures*: the design function captures `J`, and any fixed inputs like $\rho$ or the focal set, from its enclosing environment. Use a factory that takes the fixed inputs and returns the closure.

Constrain parameters inside the closure, never outside. A probability comes from `plogis(psi)`; a set of probabilities summing to one comes from a softmax with a reference category. That way `psi` can be any real vector and the optimizer never has to respect a boundary.

Here is a complete design — the record-linkage slab with a known pool distribution $\rho$ and a common rate per measure:

```
make_slab_design <- function(rho) {
  J <- length(rho)
  function(psi) {
    alpha <- plogis(psi)                          # two rates, in (0, 1)
    D1 <- diag(J) * (1 - alpha[1]) + outer(rho, rep(alpha[1], J))
    D2 <- diag(J) * (1 - alpha[2]) + outer(rho, rep(alpha[2], J))
    combine_Delta(D1, D2)
  }
}
```

Two practical notes.

**Always pass `psi_0`.** `misclassifyr()` knows starting values only for its three built-in designs, which it recognizes by a `"name"` attribute. For a custom design you must supply `psi_0` yourself. If you do not, and your closure carries no `"name"` attribute, the function errors — which is the outcome you want. If you have set a `"name"` attribute that the package does not recognize *and* omitted `psi_0`, the starting value silently stays `NA` and the optimization fails downstream with an obscure message. The simplest rule: pass `psi_0`, and do not bother setting `"name"`.

**Know what the diagonal-dominance penalty does and does not cover.** `misclassifyr()` adds a penalty, scaled by `lambda_dd`, whenever a row of $\Delta$ fails to peak at the column corresponding to its own latent value within some $Y_2$ block. Under conditional independence that is a constraint on $\Delta^{(1)}$ only: each of its columns must have its largest entry on the diagonal. $\Delta^{(2)}$ is left unconstrained, so if your second measure could plausibly mis-report more often than it reports correctly, the penalty will not stop it. The default `lambda_dd = sum(tab$n)^2` makes the $\Delta^{(1)}$ constraint effectively hard, which is what rules out the relabelled solution; lower it if your design legitimately needs to explore that region.

### The matching log prior

If you want posterior inference (`bayesian = TRUE`) you also need `log_prior_Delta`, a function of the same `psi`. It is *not* the log density of your parameters; it is the log density of a chosen prior on the **probabilities**, expressed in `psi` coordinates. Moving from probability space to unconstrained space multiplies the density by the Jacobian determinant of the link, so

$$\log p(\psi) = \log p\big(\Delta(\psi)\big) + \log \big|\det J_{\Delta}(\psi)\big|.$$

For a flat prior on a simplex reached by a softmax with a reference category, `logit_link_volume()` returns exactly that term. For a single probability reached by `plogis()`, the volume factor is `dlogis(psi, log = TRUE)`. Two lines, and you get them right by checking them numerically (below) rather than by rederiving them.

```
log_prior_slab <- function(psi) sum(dlogis(psi, log = TRUE))
```

## Step 4: verify before touching real data

Every design gets the same treatment: a structural check that costs milliseconds, then a recovery check on simulated data. Write the harness once and point it at everything.

```
check_Delta_design <- function(f, psi, J, tol = 1e-8) {
  v <- f(psi)
  stopifnot("wrong length: expected J^3"      = length(v) == J^3)
  D <- matrix(v, nrow = J)
  stopifnot("wrong shape: expected J x J^2"   = all(dim(D) == c(J, J^2)),
            "negative entries in Delta"       = all(D >= -tol),
            "rows of Delta do not sum to one" = max(abs(rowSums(D) - 1)) < tol)

  # Diagonal dominance: within each Y2 block, row y* must peak at column y*
  peaks_ok <- all(vapply(1:J, function(y2) {
    blk <- D[, ((y2 - 1) * J + 1):(y2 * J), drop = FALSE]
    all(diag(blk) >= apply(blk, 1, max) - tol)
  }, logical(1)))

  # Every parameter must actually move the matrix. A block of psi that is
  # silently ignored is the single most common defect in generated code.
  moves <- vapply(seq_along(psi), function(i) {
    psi2 <- psi; psi2[i] <- psi2[i] + 0.25
    max(abs(f(psi2) - v)) > tol
  }, logical(1))

  list(structure_ok = TRUE,
       diagonally_dominant = peaks_ok,
       inert_parameters = which(!moves))
}
```

The last check is not paranoia. This package's own history includes a released version in which the prior for the second misclassification matrix was computed from the first matrix's block of `psi` — the second block was inert, and nothing about the output looked wrong. Perturbing one coordinate at a time catches it in a second.

Run it on the slab design:

```
rho <- c(0.30, 0.25, 0.20, 0.15, 0.10)
slab <- make_slab_design(rho)
check_Delta_design(slab, psi = qlogis(c(0.2, 0.3)), J = 5)
#> $structure_ok
#> [1] TRUE
#>
#> $diagonally_dominant
#> [1] TRUE
#>
#> $inert_parameters
#> integer(0)
```

### Recovery on simulated data

Structural correctness is necessary, not sufficient. The real test is whether the estimator, fed data generated *from your design*, gets your parameters back. Two helpers do this. For the built-in shapes, `synthetic_data()` generates a tabulation directly:

```r
set.seed(1)
syn <- synthetic_data(J = 3, K = 3, I = 1, sample_size = 5000,
                      dgp_delta = "Record Linkage, independent, 10 - 30%")
head(syn$tab[[1]], 3)
#>   X Y1 Y2   n
#> 1 1  1  1 691
#> 2 2  1  1 444
#> 3 3  1  1  93
```

For a custom design you simulate from it yourself. This is six lines, and it is worth understanding because the cell ordering it produces — `X` fastest, then `Y1`, then `Y2` — is exactly the ordering `misclassifyr()` expects:

```r
simulate_tab <- function(Pi, Delta, N) {
  J <- nrow(Pi); K <- ncol(Pi)
  p <- c(t(Pi) %*% matrix(Delta, nrow = J))       # cell probabilities
  data.frame(X  = rep(1:K, J^2),
             Y1 = rep(rep(1:J, each = K), J),
             Y2 = rep(1:J, each = J * K),
             n  = as.numeric(rmultinom(1, N, p)))
}

# A diagonal-heavy latent joint distribution to simulate from
make_Pi <- function(J, K) {
  P <- outer(1:J, 1:K, function(j, k) exp(-0.5 * (j - k)^2))
  P / sum(P)
}
```

Now generate data with known $\alpha = (0.20, 0.30)$ and see whether the estimator finds them:

```r
set.seed(13)
J <- 5; K <- 5
Pi_true    <- make_Pi(J, K)
alpha_true <- c(0.20, 0.30)
tab_slab   <- simulate_tab(Pi_true, slab(qlogis(alpha_true)), N = 2e5)

fit_slab <- misclassifyr(
  tab = tab_slab, J = J, K = K,
  X_names = as.character(1:K),
  Y1_names = as.character(1:J), Y2_names = as.character(1:J),
  model_to_Delta = slab, psi_0 = c(-1, -1),
  X_vals = 1:K, Y_vals = 1:J,
  mle = TRUE, bayesian = FALSE, makeplots = FALSE
)

round(plogis(tail(fit_slab$eta_hat_mle, 2)), 3)   # should be 0.20, 0.30
#> [1] 0.200 0.298
max(abs(matrix(fit_slab$Pi_hat_mle, nrow = J) - Pi_true))
#> [1] 0.0006511507
```

If a design fails this test, it is either not identified or wrongly coded, and no amount of real data will fix it.

## Using a language model as a design assistant

Step 3 — translating a shape you have already reasoned about into indexed R code — is exactly the kind of task a large language model does well and you do slowly. Steps 1, 2, and 4 are not. Delegate accordingly.

Four things make the difference between a useful draft and a plausible mess:

- **State the layout contract explicitly.** Do not assume the assistant knows this package. Paste the row/column convention every time.
- **Give the parameterization, do not ask for one.** You decided the shape in step 2; the assistant's job is to implement it, not to choose it.
- **Ask for the closure factory, not a bare function**, so that `J` and any fixed inputs are captured.
- **Ask it to state its assumptions.** A good response flags the ones you did not pin down — boundary handling, whether the rate varies with $y^*$ — instead of silently choosing.

Below are four prompts of the kind that work, with what a good response looks like and how to check it. In every case the check is the same harness from step 4, and in every case you should run it before reading the code closely.

### Prompt 1: the record-linkage slab with a known pool

```
I am using the R package misclassifyr. I need a `model_to_Delta` function.

The contract:
- The function takes one argument, `psi`, an unconstrained numeric vector.
- It returns `c(Delta)`, a numeric vector of length J^3 holding a J x J^2
  matrix column-major.
- Rows of that matrix index the latent outcome Y*; column (y2-1)*J + y1
  holds Pr(Y1 = y1, Y2 = y2 | Y* = y*). Rows sum to one.
- Errors in the two measures are conditionally independent given Y*, so
  build two J x J matrices Delta1, Delta2 with Pr(Ym = ym | Y* = y*) in
  entry [ym, y*] (columns sum to one) and combine them.

The model: each measure is a record-linkage slab. With probability
1 - alpha_m the recorded value equals Y*; with probability alpha_m it is an
independent draw from a KNOWN probability vector rho of length J. There are
exactly two free parameters, alpha_1 and alpha_2, constrained to (0,1)
inside the function.

Write it as a factory `make_slab_design(rho)` that returns the closure.
Also give me `psi_0`, a sensible starting value for a 15% error rate.
State any assumptions you had to make.
```

**What a good response looks like.** The factory captures `rho` and derives `J <- length(rho)`; `alpha <- plogis(psi)` inside the closure; the slab built as `diag(J) * (1 - alpha) + outer(rho, rep(alpha, J))`; the combination step using the `diag(Delta2[y2, ]) %*% t(Delta1)` idiom; `psi_0 <- rep(qlogis(0.15), 2)`. A good response also *tells* you that it assumed the same `rho` for both measures and that `rho` is not renormalized after the diagonal is added — both real choices you might have wanted differently.

**How to verify.** Run `check_Delta_design()`, and then — because this design has a special case that the package already implements — check that the two agree to machine precision. `model_to_Delta_RL_ind()` builds the same slab, but parameterizes $\rho$ through a softmax rather than taking it as given, so feeding it the `psi` that encodes our $\rho$ must reproduce our matrix exactly:

```r
gen <- make_slab_design(rho)                    # the generated factory
alpha <- c(0.18, 0.24)

check_Delta_design(gen, qlogis(alpha), J = 5)
#> $structure_ok
#> [1] TRUE
#>
#> $diagonally_dominant
```

```
#> [1] TRUE
#>
#> $inert_parameters
#> integer(0)

# The same slab, expressed in model_to_Delta_RL_ind's parameterization
r <- log(rho / rho[length(rho)])[1:(length(rho) - 1)]
psi_ref <- c(r, r, rep(qlogis(alpha[1]), 5), rep(qlogis(alpha[2]), 5))

max(abs(gen(qlogis(alpha)) - model_to_Delta_RL_ind(psi_ref)))
#> [1] 1.110223e-16
```

Cross-checking against an implementation you already trust is the strongest verification available, and it is worth structuring your designs so that at least one special case reduces to something shipped.

### Prompt 2: ordinal drift

```
Same package, same contract as before (J x J^2, rows are Y*, column
(y2-1)*J + y1, rows sum to one, conditional independence).

The model: the outcome is ORDERED (occupational rank 1..J). An error moves
the recorded value at most one rank. For measure m, define probabilities
(p_down_m, p_stay_m, p_up_m) summing to one, constant across ranks. At rank
1 there is no "down" and at rank J no "up": in those columns, drop the
impossible move and renormalize the remaining two probabilities.

Parameterize with a softmax using "stay" as the reference category, so psi
has 2 entries per measure (4 total) and is unconstrained.

Write it as `make_ordinal_design(J)` returning the closure. Give me psi_0
for roughly 10% down and 10% up. State your assumptions.
```

**What a good response looks like.** An inner helper that builds one `J` by `J` matrix from `(p_down, p_up)`, a loop over columns that writes at most three entries, and explicit renormalization in the two boundary columns. `w <- exp(c(psi[1], 0, psi[2])); w <- w / sum(w)` for the softmax. The response should flag that renormalizing at the boundary raises `p_stay` there — a modelling choice, not a bug, but one you should have made knowingly.

**How to verify.** Structural check, then recovery:

```
make_ordinal_design <- function(J) {
  one_measure <- function(p_down, p_up) {
    D <- matrix(0, J, J)
    for (y in 1:J) {
      w <- c(down = if (y > 1) p_down else 0,
             stay = 1 - p_down - p_up,
             up   = if (y < J) p_up else 0)
      w <- w / sum(w)                              # renormalize at boundaries
      if (y > 1) D[y - 1, y] <- w[["down"]]
      D[y, y] <- w[["stay"]]
      if (y < J) D[y + 1, y] <- w[["up"]]
    }
    D
  }
  function(psi) {
```

```
    w1 <- exp(c(psi[1], 0, psi[2])); w1 <- w1 / sum(w1)
    w2 <- exp(c(psi[3], 0, psi[4])); w2 <- w2 / sum(w2)
    combine_Delta(one_measure(w1[1], w1[3]), one_measure(w2[1], w2[3]))
  }
}

J <- 4; K <- 4
ordinal <- make_ordinal_design(J)
psi_true <- c(log(0.10 / 0.80), log(0.10 / 0.80),   # measure 1: 10% / 10%
              log(0.05 / 0.80), log(0.15 / 0.80))   # measure 2:  5% / 15%

check_Delta_design(ordinal, psi_true, J = J)
#> $structure_ok
#> [1] TRUE
#>
#> $diagonally_dominant
#> [1] TRUE
#>
#> $inert_parameters
#> integer(0)
```

Structure is fine and every parameter moves the matrix. Now recovery:

```
set.seed(7)
Pi_true <- make_Pi(J, K)
tab_ord <- simulate_tab(Pi_true, ordinal(psi_true), N = 2e5)

fit_ord <- misclassifyr(
  tab = tab_ord, J = J, K = K,
  X_names = as.character(1:K),
  Y1_names = as.character(1:J), Y2_names = as.character(1:J),
  model_to_Delta = ordinal, psi_0 = rep(-2, 4),
  X_vals = 1:K, Y_vals = 1:J,
  mle = TRUE, bayesian = FALSE, makeplots = FALSE
)

psi_hat <- tail(fit_ord$eta_hat_mle, 4)
softmax <- function(x) { w <- exp(c(x[1], 0, x[2])); w / sum(w) }
rbind(truth     = c(softmax(psi_true[1:2]), softmax(psi_true[3:4])),
      estimated = c(softmax(psi_hat[1:2]),  softmax(psi_hat[3:4]))) |>
  round(3)
#>             [,1]  [,2] [,3]  [,4]  [,5]  [,6]
#> truth     0.100 0.800  0.1 0.050 0.800 0.150
#> estimated 0.102 0.798  0.1 0.049 0.803 0.148
```

Both triples come back. And the point of the whole exercise — the correction moves the coefficient in the right direction:

```
naive_beta <- local({
  P <- tapply(tab_ord$n, list(tab_ord$Y1, tab_ord$X), sum)
  Pi_to_beta_inner(c(P / sum(P)), 1:K, 1:J, 1)
})
c(truth     = Pi_to_beta_inner(c(Pi_true), 1:K, 1:J, 1),
  naive     = naive_beta,
  corrected = unname(Pi_to_beta(X_vals = 1:K, Y_vals = 1:J, mle = TRUE,
```

```
                                Pi_mle = fit_ord$Pi_hat_mle,
                                cov_Pi = fit_ord$cov_Pi_mle)$beta_hat_mle)) |>
  round(4)
#>     truth     naive corrected
#>    0.6687    0.6263    0.6695
```

**Prompt 3: heaping on focal values**

```
Same package, same contract.

The model: reported values HEAP on a known set of focal categories. I will
pass `focal`, an integer vector of focal category indices. With probability
h_m the reported value is snapped to the nearest focal category (ties: the
smaller index); otherwise it is recorded correctly. One parameter per
measure, so psi has length 2, constrained to (0,1) with plogis().

Note this means a focal category is never mis-reported, and that a column
whose true value is already focal is degenerate. That is intended.

Write it as `make_heaping_design(J, focal)` returning the closure. Precompute
the nearest-focal lookup outside the closure so it is not recomputed on
every likelihood evaluation. State your assumptions.
```

**What a good response looks like.** `nearest <- sapply(1:J, function(y) focal[which.min(abs(focal - y))])` computed **in the factory**, not the closure — that detail matters, because the closure is called tens of thousands of times inside `optim()`. The matrix built as `diag(J) * (1 - h)` with `h` *added* to the `nearest[y]` entry of column `y`, so that a focal column ends up degenerate at one rather than at `1 - h + h` split across two cells. A good response warns that diagonal dominance requires `h < 0.5`.

**How to verify.** The structural harness catches the dominance issue directly, so probe it at a value that should fail:

```
make_heaping_design <- function(J, focal) {
  nearest <- vapply(1:J, function(y) focal[which.min(abs(focal - y))], numeric(1))
  one_measure <- function(h) {
    D <- diag(J) * (1 - h)
    for (y in 1:J) D[nearest[y], y] <- D[nearest[y], y] + h
    D
  }
  function(psi) {
    h <- plogis(psi)
    combine_Delta(one_measure(h[1]), one_measure(h[2]))
  }
}

J <- 6; K <- 6
heaping <- make_heaping_design(J, focal = c(1, 3, 6))

check_Delta_design(heaping, qlogis(c(0.25, 0.40)), J = J)$diagonally_dominant
#> [1] TRUE
check_Delta_design(heaping, qlogis(c(0.65, 0.65)), J = J)$diagonally_dominant
#> [1] FALSE
```

The design is legitimate below one half and not above it, exactly as the response should have warned. Recovery at an admissible value:

```r
set.seed(11)
Pi_true <- make_Pi(J, K)
h_true  <- c(0.25, 0.40)
tab_hp  <- simulate_tab(Pi_true, heaping(qlogis(h_true)), N = 2e5)

fit_hp <- misclassifyr(
  tab = tab_hp, J = J, K = K,
  X_names = as.character(1:K),
  Y1_names = as.character(1:J), Y2_names = as.character(1:J),
  model_to_Delta = heaping, psi_0 = c(0, 0),
  X_vals = 1:K, Y_vals = 1:J,
  mle = TRUE, bayesian = FALSE, makeplots = FALSE
)
#> Warning in estimate_misclassification(misclassification_inputs): Optimal theta
#> is inconsistent across starting locations.
#> Warning in estimate_misclassification(misclassification_inputs): The Fisher
#> information matrix is not invertible; using the Moore-Penrose inverse instead
#> -- analytical variances may be inaccurate.  Consider reporting highest
#> posterior density (HPD) sets instead.

rbind(truth = h_true, estimated = plogis(tail(fit_hp$eta_hat_mle, 2))) |> round(3)
#>             [,1]  [,2]
#> truth      0.250 0.400
#> estimated 0.254 0.399
```

Both rates come back — and note the two warnings, which are *expected here and should not alarm you.* A heaping matrix has structural zeros: a non-focal column has mass on exactly two cells and nothing anywhere else. That pushes parts of the estimated joint distribution against the boundary of the simplex, where the likelihood cannot curve, so the information matrix is singular by construction. The point estimates are unaffected; the analytical standard errors are not usable. `vignette("inference")` explains how to tell this benign case apart from a genuine identification failure, and what to report instead.

And the substantive lesson that makes heaping worth modelling separately:

```r
naive_hp <- local({
  P <- tapply(tab_hp$n, list(tab_hp$Y1, tab_hp$X), sum)
  Pi_to_beta_inner(c(P / sum(P)), 1:K, 1:J, 1)
})
c(truth     = Pi_to_beta_inner(c(Pi_true), 1:K, 1:J, 1),
  naive     = naive_hp,
  corrected = unname(Pi_to_beta(X_vals = 1:K, Y_vals = 1:J, mle = TRUE,
                                Pi_mle = fit_hp$Pi_hat_mle,
                                cov_Pi = fit_hp$cov_Pi_mle)$beta_hat_mle)) |>
  round(4)
#>     truth     naive corrected
#>    0.8369    0.8772    0.8386
```

The naive estimate is *larger* than the truth. Misclassification does not always attenuate: heaping pulls observations toward a small number of focal categories, which here happen to be spread across the range, and the induced error is correlated with the truth in a way that inflates the slope. Anyone who assumed "measurement error biases towards zero, so my estimate is a lower bound" would have drawn the wrong conclusion.

**Prompt 4: the matching log prior**

```
Same package. I have this `model_to_Delta` closure:

    function(psi) {
      w1 <- exp(c(psi[1], 0, psi[2])); w1 <- w1 / sum(w1)
      w2 <- exp(c(psi[3], 0, psi[4])); w2 <- w2 / sum(w2)
      combine_Delta(one_measure(w1[1], w1[3]), one_measure(w2[1], w2[3]))
    }

I want `log_prior_Delta(psi)`: the log density, in psi coordinates, of a
FLAT prior on the underlying probability simplices. That means the log
absolute determinant of the Jacobian of the map from psi to probabilities,
summed over the two independent softmax blocks.

misclassifyr exports `logit_link_volume(x)` which returns exactly this
volume term for one softmax-with-reference block, up to an additive
constant that does not depend on x. Use it. Do not rederive the Jacobian.

Also tell me how I can check the answer numerically.
```

**What a good response looks like.** Three lines — `logit_link_volume(psi[1:2]) + logit_link_volume(psi[3:4])` — plus, in answer to the last sentence, a numerical check comparing `logit_link_volume()` against `log(abs(det(numDeriv::jacobian(...))))` on the free coordinates of the softmax. Be suspicious of any response that computes the prior from `psi[1:2]` twice; that is the exact bug this package once shipped, and it is an easy one to generate.

**How to verify.** Do the numerical check the response should have proposed:

```
log_prior_ordinal <- function(psi) {
  logit_link_volume(psi[1:2]) + logit_link_volume(psi[3:4])
}

# The free coordinates of a softmax with a reference category
softmax_free <- function(x) { p <- exp(c(x, 0)); p <- p / sum(p); p[-length(p)] }

x <- c(-1.2, 0.4)
numeric_logdet <- log(abs(det(numDeriv::jacobian(softmax_free, x))))
c(numeric   = numeric_logdet,
  from_pkg  = logit_link_volume(x) - lfactorial(3))   # constant removed
#>   numeric  from_pkg
#> -3.881369 -3.881369
```

They agree to machine precision. And the inertness check applies to priors too:

```
base <- log_prior_ordinal(c(0, 0, 0, 0))
vapply(1:4, function(i) {
  psi <- c(0, 0, 0, 0); psi[i] <- 0.5
  abs(log_prior_ordinal(psi) - base) > 1e-10
}, logical(1))
#> [1] TRUE TRUE TRUE TRUE
```

All four `TRUE`: every coordinate reaches the prior.

### Stacking: enriching the outcome with what the linker ignored

There is one more design idea that is less about the shape of $\Delta$ than about what you feed it, and it is often the highest-return change available.

Return to the record-linkage slab. A failed link draws a phantom from the pool of candidates the algorithm was choosing among. Which variables that pool is restricted on turns out to matter twice over, in opposite-looking ways that call for the same fix.

**If the algorithm ignored a variable, that variable is free information.** Linking algorithms typically match on name, birth year, and birthplace, and routinely ignore county or parish of *residence*, because people move. That makes residence enormously informative about whether a link is real: a correctly linked man is usually still somewhere plausible, whereas a phantom lands wherever the name pool happens to sit. Enriching the outcome with a variable the linker never used — pairing occupation with county, say — sharpens the contrast between correct and failed links well beyond what either variable does alone. You are not adding a control; you are adding a dimension along which the error is more visible.

**If the algorithm used a variable, ignoring that is a bias.** Suppose the linker *did* match on birthplace. Then a phantom is a same-birthplace person, and a $\rho$ estimated from the unconditional distribution of birthplace is too diffuse. Phantoms that agree with the truth on birthplace look like correct links, and $\hat{\alpha}$ is biased downward.

Both point to the same recipe: split the tabulation by the variable, and estimate a **shared** error rate across the resulting cells. That is `misclassifyr_rl_em_stacked()`. It takes a list of tabulations, holds $\alpha_1, \alpha_2$ common across cells, and lets the phantom distribution $\rho_c$ and the joint distribution $\Pi_c$ vary freely within each — so the error rate is estimated from all the data at once, while the phantom pool is allowed to be as local as it really is.

Here is the failure it prevents. Four birthplace cells; the linker matched on birthplace, so a failed link draws a phantom from the *same* cell:

```
set.seed(41)
C <- 4; J <- 20; N <- 8e4; alpha_true <- 0.25

cell <- sample.int(C, N, replace = TRUE)
base <- (cell - 1) * 5                                    # 5 counties per cell
xi   <- base + sample.int(5, N, replace = TRUE)           # father's county
j    <- ifelse(runif(N) < 0.8, xi,
               base + sample.int(5, N, replace = TRUE)) # son's true county
y1   <- ifelse(runif(N) < alpha_true,
               base + sample.int(5, N, replace = TRUE), j)
y2   <- ifelse(runif(N) < alpha_true,
               base + sample.int(5, N, replace = TRUE), j)
df   <- data.frame(cell = cell, X = xi, Y1 = y1, Y2 = y2)

tabulate_cells <- function(d) {
  aggregate(list(n = rep(1, nrow(d))),
            by = list(X = d$X, Y1 = d$Y1, Y2 = d$Y2), FUN = sum)
}

pooled  <- misclassifyr_rl_em(tabulate_cells(df), J = J, K = J)
stacked <- misclassifyr_rl_em_stacked(lapply(split(df, df$cell), tabulate_cells),
                                      J = J, K = J)

rbind(pooled  = pooled$alpha,
      stacked = stacked$alpha) |> round(3)
#>         alpha1 alpha2
```

```
#> pooled   0.186  0.189
#> stacked  0.248  0.250
```

The pooled estimator, which implicitly treats the phantom pool as the whole country, understates the false-link rate by about a quarter of its value. The stacked estimator recovers it. If you then use $\hat{\alpha}$ to correct a downstream coefficient, that error propagates directly into the correction.

The practical recipe:

1. Find a variable that the linking algorithm did **not** use, or used differently from how you will condition on it. County or parish of residence is the usual candidate in historical data; linkers routinely ignore it because people move.
2. Tabulate separately within each value of that variable.
3. Estimate with `misclassifyr_rl_em_stacked()`, which shares $\alpha$ and lets everything else vary by cell.
4. Check that $\hat{\alpha}$ is stable as you coarsen or refine the cells. If it drifts a lot, the conditioning variable is doing something other than what you think.

The per-cell output carries everything you need for the downstream step:

```
str(stacked$cells[[1]], max.level = 1)
#> List of 4
#>  $ rho1: num [1:20] 0.207 0.194 0.2 0.209 0.19 ...
#>  $ rho2: num [1:20] 0.199 0.203 0.193 0.204 0.201 ...
#>  $ Pi  :'data.frame':    25 obs. of  3 variables:
#>  $ N   : num 19875
```

`vignette("sparse-and-large")` picks up from here: how to run this when the conditioning variable has hundreds of values and the outcome thousands, and how to hold $\hat{\alpha}$ fixed from a coarse first stage while estimating a fine-grained $\Pi$ in a second.